

Day 46 — Line-by-Line Syntax Explanation

day46_learning_rate_schedules.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 5 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula, a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system.

MATH box legend: $\times$ = multiply lr_0/lr_{min} = schedule endpoints $\text{floor}(x)$ = floor division π = pi m = dataset size

Segment 0 — Imports and Global Setup

```
import pickle
import time

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

np.random.seed(42)
```

Standard imports, plus `matplotlib.use("Agg")` before importing `pyplot` -- this tells `matplotlib` to render to files without needing a display, required in this sandboxed environment. The global `np.random.seed(42)` is a safety net, not the real source of reproducibility -- every network and dataset generator below uses its OWN local `np.random.RandomState(seed)`, so results don't actually depend on this global call.

Segment 1 — Shared Helpers (Reused Verbatim from Days 39-45)

```
def relu(z): return np.maximum(0, z)
def drelu(z): return (z > 0).astype(z.dtype)
def tanh(z): return np.tanh(z)
def dtanh(z): return 1 - np.tanh(z) ** 2
def softmax(z): ...
def one_hot(y, n_classes): ...
def cross_entropy_loss(probs, y_onehot): ...
def he_scale(n_in): return np.sqrt(2.0 / n_in)
def conv2d_forward_mc(X, W, b): ...
def conv2d_backward_mc(dZ, X, W): ...
def maxpool_forward(A, size=2, stride=2): ...
def maxpool_backward(dOut, A, idx_cache, size=2, stride=2): ...
def make_junction_dataset_rgb(...): ...
class TwoBlockConvNetRGB: ...
class WideTwoBlockConvNetRGB(TwoBlockConvNetRGB): pass
```

None of this changed today -- it's copied character-for-character from Day 45 (itself copied from Day 43/44).

Today's whole lesson is about a NEW way to call these networks' existing `.backward()` method, not about changing what that method computes. The one detail that matters for everything that follows:

`TwoBlockConvNetRGB.backward()` ends with `p -= self.lr * g` for every parameter `p` -- `self.lr` is a plain mutable attribute, read fresh on every call. That is the entire hook a schedule needs.

Segment 2 — Learning-Rate Schedules From Scratch

MATH

step decay: $lr(epoch) = lr_0 \times decay_factor^{\lfloor epoch / decay_every \rfloor}$

cosine annealing: $lr(epoch) = lr_min + \frac{1}{2}(lr_0 - lr_min) \times (1 + \cos(\pi \times epoch / total_epochs))$

```
def constant_schedule(lr0):
    def sched(epoch):
        return lr0
    return sched

def step_decay_schedule(lr0, decay_factor, decay_every):
    def sched(epoch):
        return lr0 * (decay_factor ** (epoch // decay_every))
    return sched

def cosine_annealing_schedule(lr0, lr_min, total_epochs):
    def sched(epoch):
        return lr_min + 0.5 * (lr0 - lr_min) * (
            1 + np.cos(np.pi * epoch / total_epochs))
    return sched
```

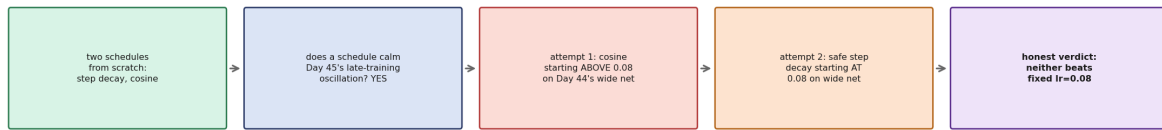
Each schedule is a CLOSURE: a function that returns another function. Calling `step_decay_schedule(0.08, 0.5, 100)` doesn't compute anything yet -- it returns a small `sched(epoch)` function that remembers `lr0=0.08`, `decay_factor=0.5`, and `decay_every=100` in its closure, and computes the actual `lr` only when later called with a specific epoch number. This is why every schedule has the SAME signature (one argument: epoch) despite needing different numbers of configuration parameters -- the configuration is baked in at construction time, not passed on every call.

```
def demo_schedule_shapes():
    const = constant_schedule(0.08)
    step = step_decay_schedule(lr0=0.08, decay_factor=0.5, decay_every=100)
    cos = cosine_annealing_schedule(lr0=0.16, lr_min=0.0005, total_epochs=400)
    for e in [0, 50, 99, 100, ...]:
        print(f"{e:>7}{const(e):>16.5f}{step(e):>13.5f}{cos(e):>10.5f}")
```

This demo calls all three schedule functions directly with NO training happening at all -- it's purely evaluating the formulas above at several checkpoint epochs, to build intuition for the shape of each curve before using them for anything. The integer `//` in `step_decay`'s `epoch // decay_every` is floor division: it stays 0 for epochs 0-99, becomes 1 for epochs 100-199, etc, which is exactly what produces the staircase shape.

SYSTEM DIAGRAM

Day 46 pipeline: from schedule mechanics to an honest verdict on Day 44's wide network



The day's full pipeline: schedules from scratch, calming Day 45's oscillation, two attempts at Day 44's wide network, honest verdict.

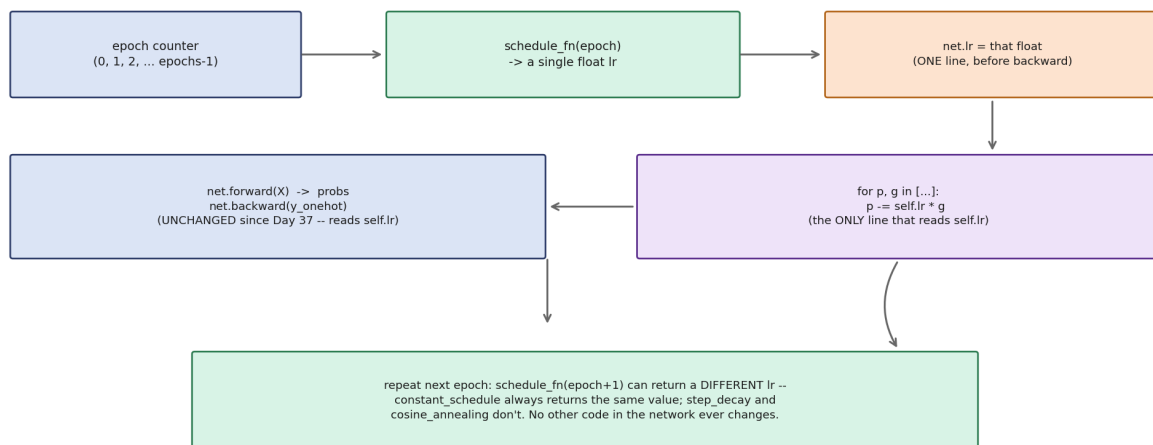
Segment 3 — A Generic Scheduled Training Loop

```
def train_scheduled(net, X, y, epochs, schedule_fn, n_classes=4):
    y_onehot = one_hot(y, n_classes)
    losses, lrs = [], []
    for epoch in range(epochs):
        net.lr = schedule_fn(epoch)
        probs = net.forward(X)
        losses.append(cross_entropy_loss(probs, y_onehot))
        net.backward(y_onehot)
        lrs.append(net.lr)
    return losses, lrs
```

Compare this to every prior day's plain training loop -- the **ONLY** new line is `net.lr = schedule_fn(epoch)`, placed before `forward()` (though it would work identically placed right before `backward()`, since `lr` is only ever **READ** inside `backward`). Passing `schedule_fn = constant_schedule(0.08)` makes this function behave **EXACTLY** like every network's old fixed-`lr` loop, because `constant_schedule(0.08)` returns a function that always returns 0.08 regardless of epoch -- a `schedule` is a strict generalization, not a separate code path.

SYSTEM DIAGRAM

Where a learning-rate schedule actually plugs in: one attribute, one call site



The full injection point: epoch counter feeds `schedule_fn`, which produces one float that overwrites `net.lr`, read once inside `backward()`.

```
def noise_std_metric(step_losses, window="second_half"):
    losses_arr = np.asarray(step_losses)
    if window == "second_half":
        tail = losses_arr[len(losses_arr) // 2:]
    else:
        tail = losses_arr[-window:]
    if len(tail) < 2:
        return 0.0
    return float(np.std(np.diff(tail)))
```

Reused verbatim from Day 45: measures how NOISY a loss curve is by taking the standard deviation of consecutive step-to-step differences (`np.diff`), over either the second half of training or just the last K steps. This is the metric used throughout today's lesson to quantify how much a schedule 'calms' an oscillating loss curve.

Segment 4 — Does a Schedule Calm Day 45's Oscillation?

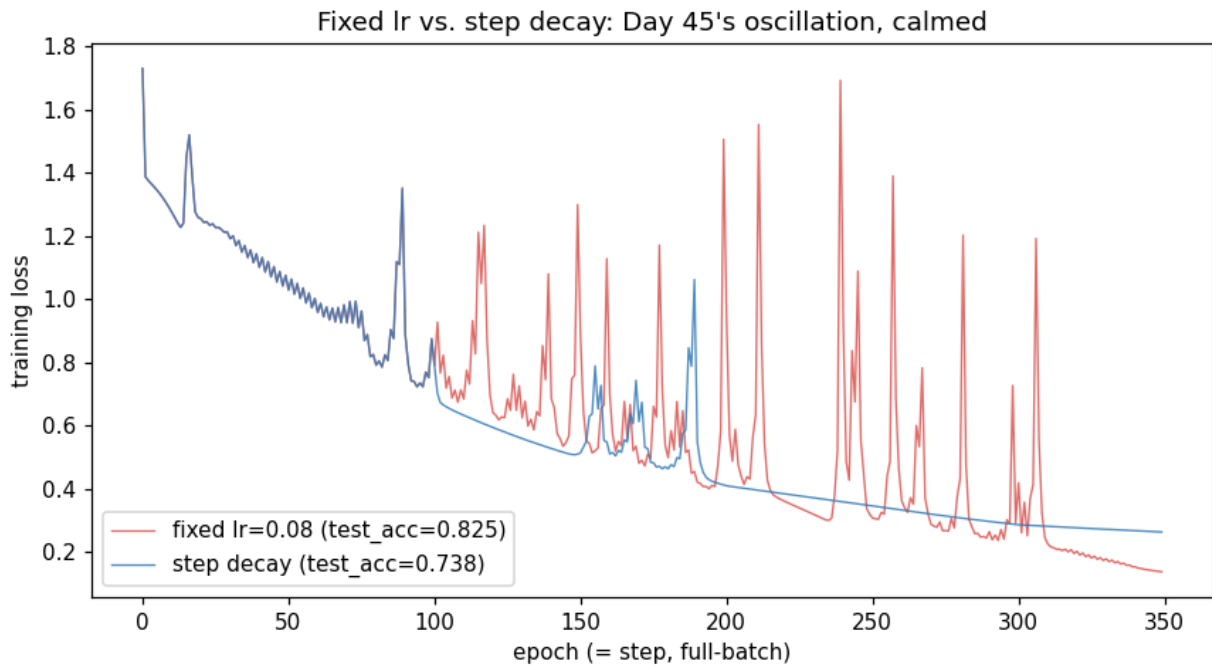
```
X_train, y_train = make_junction_dataset_rgb(
    n_per_class=50, size=30, arm=8, noise=0.4, channel_noise=0.15,
    center_jitter=2, seed=42)
X_test, y_test = make_junction_dataset_rgb(
    n_per_class=20, size=30, arm=8, noise=0.4, channel_noise=0.15,
    center_jitter=2, seed=999)
```

This EXACT dataset convention (seed=42 for a 200-image train set, a SEPARATE seed=999 generation for an 80-image test set) is what Day 44 and Day 45 both actually used -- not a single generation with a held-out slice. Getting this wrong on the first attempt at building this exact segment (assuming a different, made-up convention) produced numbers that didn't match Day 45's own documented results at all, caught only by comparing against those real, already-verified numbers. Always re-check a prior day's ACTUAL dataset-generation calls rather than assuming a convention from memory.

```
net_fixed = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                               lr=0.08, seed=42)
losses_fixed, _ = train_scheduled(net_fixed, X_train, y_train, epochs=350,
                                  schedule_fn=constant_schedule(0.08))
...
net_decay = TwoBlockConvNetRGB(f1=3, f2=6, k=3, img_size=30, hidden=8,
                               lr=0.08, seed=42)
sched = step_decay_schedule(lr0=0.08, decay_factor=0.5, decay_every=100)
losses_decay, lrs_decay = train_scheduled(net_decay, X_train, y_train,
                                           epochs=350, schedule_fn=sched)
```

Two SEPARATE networks, each freshly constructed with the same seed=42 (so both start from IDENTICAL initial weights) -- one trained with a constant schedule (reproducing Day 45's original fixed-lr run), one with step decay. This side-by-side identical-start comparison is what makes the noise_std difference between them attributable to the SCHEDULE alone, not to any other confound.

SYSTEM DIAGRAM



Real loss curves: fixed lr=0.08 (red) keeps spiking past epoch 300; step decay (blue) smooths out almost entirely once decay begins.

Segment 5 — Revisiting Day 44's Wide Network

```
X_train, y_train = make_junction_dataset_rgb(
    n_per_class=50, size=30, arm=8, noise=0.4, channel_noise=0.15,
    center_jitter=2, seed=42)
X_test, y_test = make_junction_dataset_rgb(
    n_per_class=20, size=30, arm=8, noise=0.4, channel_noise=0.15,
    center_jitter=2, seed=999)

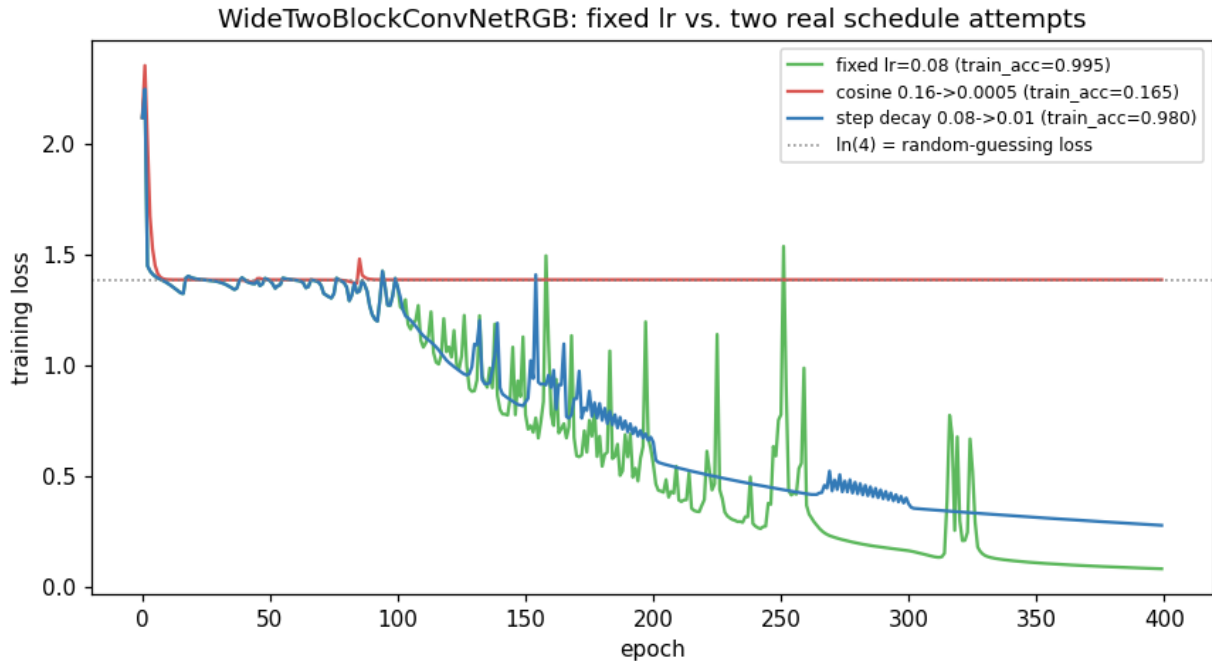
net_base = WideTwoBlockConvNetRGB(f1=6, f2=12, k=3, img_size=30, hidden=8,
    lr=0.08, seed=42)
losses_base, _ = train_scheduled(net_base, X_train, y_train, epochs=400,
    schedule_fn=constant_schedule(0.08))
```

The SAME Day 44 dataset convention as Segment 4's own generation calls (not a coincidence -- both segments needed it, since both are directly comparing against Day 44/45's real numbers). This baseline reproduction is verified against Day 44's own documented result (train_acc=0.995, test_acc=0.537) inside the actual script -- see the match = `abs(train_acc_base - 0.995) < 0.001` check in the real source, which prints whether it matches before continuing.

```
net_cos = WideTwoBlockConvNetRGB(f1=6, f2=12, k=3, img_size=30, hidden=8,
    lr=0.08, seed=42)
cos_sched = cosine_annealing_schedule(lr0=0.16, lr_min=0.0005,
    total_epochs=400)
losses_cos, lrs_cos = train_scheduled(net_cos, X_train, y_train,
    epochs=400, schedule_fn=cos_sched)
...
probs_cos = net_cos.forward(X_train)
```

lr0=0.16 is DOUBLE Day 44's safe fixed rate of 0.08 -- an intentionally aggressive choice, motivated by the idea that a schedule's later decay might make an otherwise-risky high starting rate safe. Calling `net_cos.forward(X_train)` again AFTER training finishes (not during) is how the script inspects the network's final predicted probabilities directly, to diagnose exactly what kind of failure produced the flat loss curve.

SYSTEM DIAGRAM



All three configs on WideTwoBlockConvNetRGB: green (fixed) descends steadily; blue (step decay) close behind but higher; red (cosine) flatlines exactly at $\ln(4)$ after a brief early dip.

```
net_step = WideTwoBlockConvNetRGB(f1=6, f2=12, k=3, img_size=30, hidden=8,
                                  lr=0.08, seed=42)
step_sched = step_decay_schedule(lr0=0.08, decay_factor=0.5,
                                 decay_every=100)
losses_step, lrs_step = train_scheduled(net_step, X_train, y_train,
                                         epochs=400, schedule_fn=step_sched)
```

The safer counterpart to the cosine attempt -- $\text{lr}_0=0.08$ NEVER exceeds Day 44's own proven-stable rate, only decays below it. Both wide-network attempts (this one and the cosine one above) reuse the exact same `train_scheduled` function from Segment 3 -- the only thing that differs between all three wide-network configs (base, cosine, step) is which `schedule_fn` gets passed in.

Segment 6 — What This Maps To in PyTorch

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.08)
step_sched = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=100, gamma=0.5)
cos_sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=400, eta_min=0.0005)

for epoch in range(epochs):
    train_one_epoch(...)
    step_sched.step()
```

`StepLR(optimizer, step_size=100, gamma=0.5)` is PyTorch's built-in equivalent of this lesson's `step_decay_schedule(lr0, decay_factor=0.5, decay_every=100)` -- `gamma` is `decay_factor`, `step_size` is `decay_every`, and `lr0` comes from whatever `lr` the optimizer itself was constructed with. `.step()` is called once per epoch, exactly like this lesson calling `schedule_fn(epoch)` once per epoch inside `train_scheduled`.

Quick Syntax Glossary — New or Reinforced Constructs

`def sched(epoch): ... / return sched` — A closure -- an inner function that "remembers" the outer function's arguments (`lr0`, `decay_factor`, etc.) after the outer function returns. This is what lets `step_decay_schedule(0.08, 0.5, 100)` be called ONCE to configure a schedule, then the returned `sched(epoch)` called many times with just an epoch number.

`epoch // decay_every` — Floor (integer) division -- discards the remainder, producing a step-function shape (0 for a whole range of epochs, then 1, then 2, ...) rather than a smooth ramp.

`decay_factor ** (epoch // decay_every)` — Python's exponentiation operator, applied to an integer power computed by the floor-division above -- this is what turns "halve every 100 epochs" into an actual multiplier at any given epoch.

`np.cos(np.pi * epoch / total_epochs)` — NumPy's cosine function evaluated over the range `[0, pi]` as `epoch` goes from 0 to `total_epochs`, producing a value from 1 down to -1 -- the raw shape that `cosine_annealing_schedule` rescales into the `[lr_min, lr0]` range.

`net.lr = schedule_fn(epoch)` — Reassigning a plain Python instance attribute -- no special API, no subclassing, no registration. This works because `backward()` was always written to read `self.lr` fresh on every call rather than caching it.

`abs(train_acc_base - 0.995) < 0.001` — A floating-point equality check done with a small tolerance rather than `==` -- standard practice for comparing computed floats, and used here as a real, printed sanity check that this script's baseline reproduction matches Day 44's independently-documented number.

End of Day 46 syntax reference. For the concepts these lines implement, see the main Day 46 study guide PDF.